

REPORT ON PROJECT: ATM MACHINE

Introduction:

The project aims to simulate the functionalities of an ATM system. It incorporates fundamental programming concepts such as object-oriented programming (OOP), file handling, and user interaction. The project is structured into two primary files: “**atm.h**”, which contains the implementation of the core classes and functions, and “**ATM.cpp**”, which serves as the main driver program.

Objective

The goal of this project is to create a user-friendly, secure, and functional ATM system where users can:

1. Create new accounts.
2. Log into existing accounts.
3. Perform financial transactions such as checking balance, withdrawing funds, and transferring money.

Functionalities

1. Account Management:

- **Create Account:** Users can create an account by entering details like name, PIN, phone number, CNIC, and address.
- **Login:** Users can securely log in using their ID and PIN.
- **Account Blocking:** Accounts are blocked after three consecutive failed login attempts.

2. Transactions:

- **Deposit Money:** Users can add funds to their account.
- **Withdraw Money:** Users can withdraw funds, provided the account balance is sufficient.
- **Transfer Funds:** Users can transfer money to other accounts by entering the recipient's ID.

3. Security Features:

- Four-digit PIN system for secure login.
- Account locking after multiple failed attempts.

4. Data Persistence:

- Account data is stored in a text file (**DATABASE.txt**) and synchronized during program execution for preservation and future use.

Implementation Details

1. Header File (**atm.h**):

- Contains the **account** class, which encapsulates attributes like **name**, **id**, **balance**, and **pin**.
- Includes helper functions for ID generation, account creation, deposit, withdrawal, and file synchronization.
- Implements methods to handle input/output operations, manage account attempts, and block accounts when necessary.

2. Driver File (**ATM.cpp**):

- Handles program flow by calling functions from the header file.
- Provides an interface for account creation and login.
- Manages user interaction through menu-driven options for balance checking, withdrawal, and fund transfers.

3. File Handling:

- Data is read from and written to **DATABASE.txt** to ensure the persistence of account information across program runs.

Code Highlights

1. Secure PIN Input: PINs are entered using **getch()** to hide the input, enhancing security.

2. Dynamic Account Management:

- Accounts are stored in a vector of **account** objects, allowing for efficient management of user data.

3. Real-Time Synchronization:

- Account data is updated in the file after every significant operation to maintain consistency.

Limitations

- The program is only supported by **C++17**, which limits its portability to systems or compilers that do not support this version or lower.